

AI-Enabled Quality Gates in CI/CD

José Santos^{1*}, Filipe Mesquita^{1†} and José Vasco^{1†}

^{1*}Departamento de Informática, Universidade da Beira Interior,
Caminho do Biribau, Covilhã, 6201-001, Castelo Branco, Portugal.

¹Departamento de Informática, Universidade da Beira Interior,
Caminho do Biribau, Covilhã, 6201-001, Castelo Branco, Portugal.

¹Departamento de Informática, Universidade da Beira Interior,
Caminho do Biribau, Covilhã, 6201-001, Castelo Branco, Portugal.

*Corresponding author(s). E-mail(s): j.pedro.santos@ubi.pt;
Contributing authors: filipe.mesquita@ubi.pt; jose.m.vasco@ubi.pt;

[†]Estes autores contribuíram de forma igual para este trabalho.

Abstract

Nos últimos anos, a Inteligência Artificial tem vindo a ser integrada em diversas atividades da Engenharia de *Software*, nomeadamente na garantia da qualidade e na automatização de processos de tomada de decisão. Este projeto analisa a utilização de *quality gates* suportados por Inteligência Artificial em ambientes CI/CD. O principal objetivo consiste em avaliar o impacto destes mecanismos no processo de integração de código, recorrendo a uma abordagem combinada entre automação inteligente e supervisão humana. Para tal, foi desenvolvido um ambiente experimental de simulação capaz de reproduzir diferentes cenários de integração através de pedidos de *merge*. O sistema integra testes unitários, *fuzz testing*, *mutation testing*, análise estática de código e um mecanismo de avaliação de risco responsável por suportar a tomada de decisão automática. Durante o estudo foram recolhidos e analisados dados relacionados às decisões produzidas pela Inteligência Artificial, às intervenções humanas, à frequência de *overrides* e à ocorrência de defeitos não detetados durante o processo de validação. Os resultados obtidos demonstram que os *AI Quality Gates* podem contribuir para aumentar a eficácia dos processos de validação em *pipelines* CI/CD, reduzindo o risco de integração de código defeituoso. Os resultados evidenciam igualmente a importância da supervisão humana como mecanismo complementar para reforçar a fiabilidade e a robustez das decisões automatizadas.

Keywords: Qualidade de *Software*, Inteligência Artificial, *Quality Gates*, CI/CD, *Human-in-the-Loop*, *Override*, *Fuzz Testing*, *Unit Testing*, *Mutation Testing*

1 Introdução

A crescente adoção de práticas *DevOps* e de *pipelines* de *Continuous Integration e Continuous Delivery* (CI/CD) tem permitido acelerar o desenvolvimento e a entrega de *software*. Neste contexto, a garantia da qualidade assume um papel fundamental, uma vez que alterações frequentes ao código exigem mecanismos eficientes de validação antes da integração e disponibilização de novas funcionalidades.

Recentemente, ferramentas baseadas em Inteligência Artificial (IA) começaram a ser integradas nos processos de desenvolvimento de *software*. Estas oferecem capacidades como a análise automática de código, detecção de defeitos, avaliação de risco e apoio à tomada de decisão.

O presente projeto enquadra-se no tema "AI-Enabled Quality Gates in CI/CD" e tem como objetivo analisar o impacto da utilização de mecanismos de controlo de qualidade assistidos por IA em *pipelines* CI/CD. Pretende-se comparar abordagens manuais e automatizadas, avaliando aspetos como a velocidade de execução, a qualidade das decisões tomadas e a confiança dos utilizadores nos resultados produzidos.

Para atingir estes objetivos, será desenvolvido e configurado um ambiente de integração contínua onde serão implementados diferentes tipos de *quality gates*. Posteriormente, serão recolhidos e analisados dados relacionados com aprovações, rejeições, sobreposições (*overrides*) e possíveis falhas não detetadas durante o processo.

2 Ferramentas e Tecnologias Utilizadas

2.1 Introdução

Neste projeto foram integradas diversas ferramentas de desenvolvimento, análise estática, testes dinâmicos e engenharia de dados. A seleção destas tecnologias baseou-se em critérios de robustez, interoperabilidade e ampla adoção na indústria de *software*, sendo essenciais para a sustentação e automação do processo de garantia da qualidade através de *Quality Gates*.

2.2 Python

O *Python* (na sua versão 3) foi adotado como a principal linguagem de desenvolvimento do ecossistema experimental. A sua sintaxe limpa, legibilidade e o ecossistema maduro de bibliotecas científicas e de engenharia de *software* tornam-no a escolha ideal para o desenvolvimento de *pipelines* de automação, análise estatística e Inteligência Artificial. No âmbito deste trabalho, a linguagem foi empregue tanto no desenvolvimento das aplicações de teste (*Calculator*, *Password Validation* e *Loan Approval*) como na codificação dos motores de cálculo de risco e orquestração do pipeline de simulação.

2.3 Visual Studio Code

O *Visual Studio Code* (VS Code) serviu como o Ambiente de Desenvolvimento Integrado (IDE) unificado para a equipa. A sua arquitetura leve, combinada com extensões específicas para o ecossistema *Python*, gestão de ambientes virtuais e depuração avançada de código, permitiu otimizar o fluxo de trabalho. A integração nativa com

terminais POSIX e ferramentas de controlo de versões mitigou fricções operacionais durante o ciclo de desenvolvimento.

2.4 GitHub

O *GitHub* foi utilizado como a plataforma centralizada de alojamento do repositório de código fonte, suportada pelo sistema de controlo de versões distribuído *Git*. A ferramenta desempenhou um papel crucial na governação do código através do rastreamento de alterações, gestão de ramificações (*branches*) e simulação conceptual do fluxo de integridade associado à submissão e revisão de *Pull Requests* e *Merge Requests*.

2.5 Flake8

O *Flake8* foi integrado no pipeline para atuar como o motor de análise estática de código (*linting*). Esta ferramenta agrega as capacidades de verificação de estilo do *pycodestyle* (garantindo a conformidade estrita com a norma de estilo PEP 8), análise de erros sintáticos com o *pyflakes* e verificação de complexidade estrutural preliminar. Os desvios detetados pelo *Flake8* são reportados sob a forma de contagem absoluta de erros de *lint*, penalizando o score de risco do commit.

2.6 Radon

A ferramenta *Radon* foi incorporada para a extração quantitativa de métricas de complexidade de código fonte. O *Radon* analisa a estrutura de controlo abstrata do código em *Python* e calcula a **Complexidade Ciclomática de McCabe**, medindo o número de caminhos linearmente independentes presentes nos fluxos algorítmicos. Esta métrica permite ao pipeline isolar e sinalizar funções com excesso de ramificações condicionais (*if-else*), que aumentam a probabilidade de introdução de defeitos.

2.7 Pandas

A biblioteca *pandas* foi utilizada como a infraestrutura de engenharia de dados interna do projeto. Através das suas estruturas de dados tabulares de alto desempenho (*DataFrames*), o *pandas* permitiu consolidar de forma síncrona os dados gerados pelo *Metrics Engine*, agregando as saídas heterogêneas das fases de teste, decisões preditivas da Inteligência Artificial e ações de intervenção humana para posterior exportação e auditoria estruturada.

Table 1 Resumo das tecnologias utilizadas no projeto.

Tecnologia	Categoria	Função Principal
Python	Linguagem	Implementação do sistema
Visual Studio Code	IDE	Desenvolvimento e depuração
GitHub	Versionamento	Gestão do repositório
Flake8	Análise Estática	Deteção de erros de código
Radon	Métricas	Complexidade ciclomática
Pandas	Engenharia de Dados	Processamento de métricas

3 Metodologia e Implementação

3.1 Introdução

Este capítulo descreve a metodologia detalhada e a arquitetura do sistema experimental desenvolvido neste projeto. O objetivo principal consistiu em projetar e avaliar um *pipeline* de *Continuous Integration* e *Continuous Delivery* (CI/CD) aumentado com mecanismos de decisão autônoma e semicontrolada, designados por *AI Quality Gates*, operando em ambiente de supervisão humana (*Human-in-the-Loop*).

3.2 Metodologia Experimental

A abordagem adotada neste projeto baseia-se numa estratégia de **simulação controlada**. Em vez de utilizar históricos de repositórios reais, que costumam ser incompletos e desorganizados, o ambiente experimental utiliza o *script generate_data.py*. Este módulo gera automaticamente os dados de submissão de código através de regras lógicas bem definidas. Esta escolha garante o controlo total sobre os testes, permitindo repetir as experiências e garantir que os dados recolhidos são de confiança.

Cada *merge request* criado recebe uma classificação real (*Merge Final Decision*), que pode ser **GOOD** (código bom) ou **BAD** (código mau/com erros). Esta classificação funciona como a resposta correta do sistema, indicando se o código enviado é estável ou se contém erros de lógica, problemas de segurança ou falhas na estrutura.

A grande vantagem desta abordagem é a capacidade de criar e avaliar o comportamento do sistema perante erros controlados. Isto permite perceber com clareza a eficácia de cada (*quality gate*) que foi implementado no processo.

Principais vantagens da metodologia adotada:

- **Controlo Total dos Cenários:** Permite definir com precisão a quantidade e a gravidade dos commits que contêm erros.
- **Reprodutibilidade dos Testes:** Garante que os testes correm sempre da mesma forma, eliminando atrasos com a internet ou tempos de espera variáveis.
- **Rapidez de Execução:** Possibilidade de simular e avaliar centenas de integrações completas em poucos segundos.
- **Análise Isolada:** Capacidade de medir como o componente de análise de risco (*Risk Engine*) reage quando uma métrica específica falha isoladamente (por exemplo, quando a cobertura de testes desce, mas a complexidade do código continua igual).

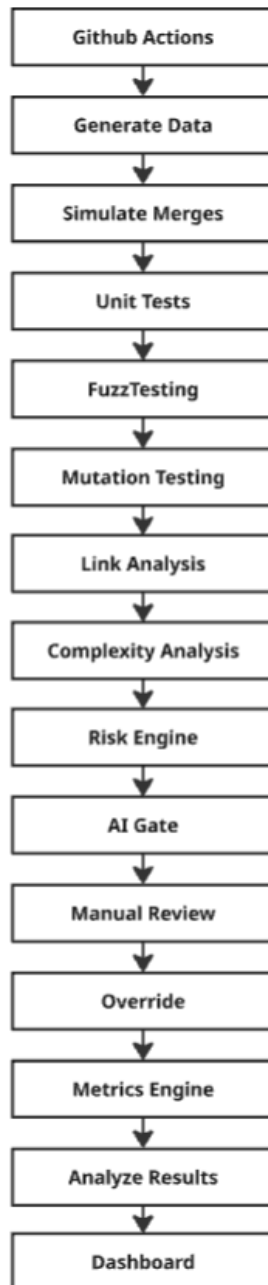


Fig. 1 Fluxo lógico e sequencial do Pipeline de Integração Simulado.

3.3 Arquitetura Geral do Sistema

A estrutura do sistema foi desenhada de forma modular, funcionando como um (*pipeline*). Cada componente trabalha de forma independente, aproveitando os resultados da fase anterior para gerar novos dados para o bloco seguinte.

O ciclo de processamento de um *merge request* segue o fluxo explicado abaixo:

1. **Ponto de Entrada:** O ficheiro *merges.json* é lido e dividido de acordo com a classificação real (*Merge Final Decision*). Se o *merge* for marcado como **BAD** (com erros), o sistema simula um cenário de falha grave, gerando automaticamente métricas fracas (como quedas na cobertura de testes e aumento da complexidade). Se for marcado como **GOOD** (sem erros), o sistema executa os testes normais sobre o código que está limpo.
2. **Camada de Extração de Métricas:** Nesta fase, o código passa por testes práticos (*Unit Testing*, *Fuzz Testing* e *Mutation Testing*) e por ferramentas de análise estática. Estas ferramentas contam os erros de estilo no código (*lint errors*) e medem a sua complexidade estrutural.
3. **Camada de Avaliação de Risco:** Esta componente reúne todas as métricas diferentes recolhidas na fase anterior e junta-as num único valor de risco, que vai de 0 a 100.
4. **Camada de Decisão Automatizada (*AI Quality Gate*):** Utiliza o valor de risco calculado para classificar automaticamente o merge em categorias de aceitação, calculando também uma margem de confiança para essa decisão.
5. **Camada de Revisão Humana e Finalização:** Simula a decisão de um programador que analisa o risco e as recomendações da IA. Se houver uma divergência entre a decisão da IA e a do humano, entra em ação um mecanismo de sobreposição (*Override System*) para definir o veredicto final.

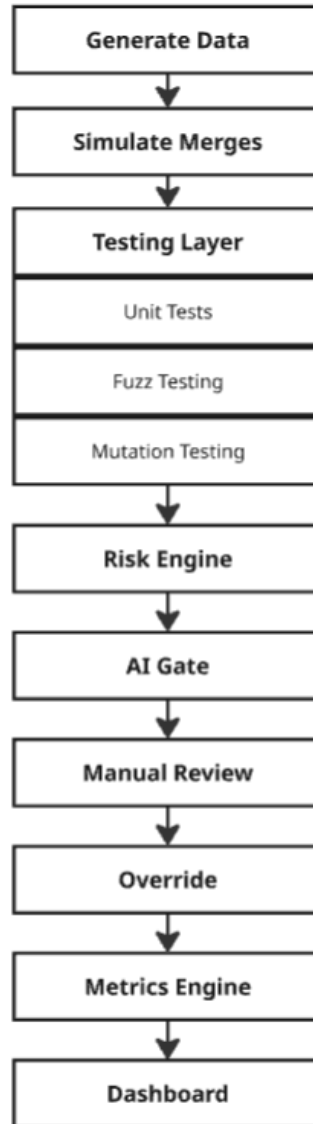


Fig. 2 Arquitetura Modular do Ecosystema de Validação de Código.

3.4 Aplicações de Teste

Para avaliar a flexibilidade do pipeline em diferentes cenários, foram desenvolvidas três aplicações independentes em *Python*. Estes sistemas simulam serviços críticos com características e objetivos muito diferentes:

3.4.1 Calculator

Esta aplicação realiza as operações matemáticas básicas: soma, subtração, multiplicação e divisão (esta última protegida contra erros de divisão por zero). Por ser um sistema simples, direto e previsível, apresenta uma complexidade muito baixa e serve como o grupo de controlo neutro do pipeline.

3.4.2 Password Validation

Implementa regras baseadas em expressões regulares (*Regex Expressions*) para verificar se as palavras-passe cumprem critérios estritos de segurança. O sistema avalia o tamanho da palavra-passe, a variedade de caracteres e valida se ela existe em dicionários de *passwords* fracas presentes num ficheiro. Introduce mais condições e desvios no código, simulando um serviço de autenticação real.

3.4.3 Loan Approval

Um sistema de avaliação de pedidos de empréstimo financeiro baseado em várias regras de decisão interligadas. Este módulo cruza cinco variáveis do utilizador (rendimento mensal, *credit score*, dívida, anos de trabalho estável e idade) para decidir o nível de risco. É a aplicação mais complexa e com mais condições lógicas, sendo ideal para testar se as nossas ferramentas conseguem medir bem a complexidade do código.

Table 2 Matriz de Caracterização Teórica das Aplicações de Teste.

Aplicação	Domínio	Complexidade	Sensibilidade Principal
Calculator	Aritmético	Muito Baixa	Falhas de Precisão Funcional
Passwords	Validação de Strings	Moderada	Inputs Inválidos (Fuzzing)
Loan Approval	Regras de Decisão	Elevada	Cobertura de Caminhos Lógicos

3.5 Estratégias de Teste e Emulação de Métricas

O comportamento dos testes divide-se em dois caminhos dentro do pipeline, garantindo o contraste necessário para avaliar o sistema:

3.5.1 Unit Testing

Serve para validar se as funções básicas do código estão a funcionar como planeado. Quando o código avaliado é bom (*GOOD*), os testes correm com dados normais para confirmar o sucesso do sistema. A sua métrica principal é a **Taxa de Sucesso dos Testes (*Pass Rate*)**, que vai de 0 a 1 (ou 0% a 100%), medindo a proporção de testes que terminaram sem falhas. Quando o código é mau (*BAD*), o pipeline simula uma quebra funcional, forçando a taxa de sucesso a descer para valores entre 35% e 60%.

3.5.2 Fuzz Testing

Envia uma quantidade massiva de dados inválidos e malformados para o sistema (como textos vazios, tamanhos exagerados e limites numéricos inválidos) para tentar provocar erros inesperados no programa. A sua métrica de saída é a **Taxa de Falhas Fuzz** (*Fuzz Failures*), que mede a resistência do sistema. Em códigos bons e estáveis esta taxa é zero, mas sobe quando o código avaliado é mau (*BAD*).

3.5.3 Mutation Testing

Avalia a qualidade e a eficácia dos próprios testes. Neste processo, são introduzidas pequenas alterações propositadas no código (mutações) para verificar se os testes conseguem dar conta do erro e eliminar o "mutante". A métrica **Mutation Score** mostra a percentagem de mutantes que foram apanhados pelos testes. Em código com problemas (*BAD*), o sistema simula que os testes falharam na deteção, reduzindo esta pontuação para valores entre 20% e 50%.

3.6 Risk Engine

O *Risk Engine* (mecanismo de avaliação de risco) funciona como um componente central que reúne todas as informações recolhidas. Ele combina as cinco métricas analisadas (*Pass Rate*, *Fuzz Failures*, *Mutation Score*, *Lint Errors* e *Cyclomatic Complexity*) e transforma-as num valor único e contínuo: o **Risk Score** (Pontuação de Risco), que varia entre 0 e 100.

O cálculo do risco dá maior peso às falhas e penalizações do código. Isto significa que sempre que há bons indicadores de qualidade (como uma taxa de sucesso dos testes alta ou uma boa pontuação de mutação), o risco desce. Pelo contrário, indicadores de degradação (como erros de estilo, alta complexidade ou falhas no teste *fuzz*) fazem o risco subir imediatamente. O valor final gerado serve de base para todas as decisões seguintes do pipeline.

Listing 1 Cálculo do Risk Score no Risk Engine

```
def compute_risk(pass_rate, fuzz_failures, mutation_score,
                 lint_errors, complexity):

    risk = 0

    if pass_rate < 0.60:
        risk += 20
    elif pass_rate < 0.80:
        risk += 10

    risk += fuzz_failures * 25

    if mutation_score < 0.5:
        risk += 25
```

```

elif mutation_score < 0.8:
    risk += 10

risk += lint_errors * 2

if complexity > 15:
    risk += 15
elif complexity > 10:
    risk += 8

return min(risk , 100)

```

O excerto apresentado corresponde à implementação da função de cálculo de risco do sistema (*Risk Engine*), responsável por agregar diferentes métricas de qualidade de *software* num único valor escalar.

Inicia-se com a avaliação da taxa de sucesso dos testes (*pass_rate*), onde valores inferiores a 0.80 introduzem penalizações progressivas, refletindo menor validação funcional do sistema. Em seguida, são considerados os resultados do *fuzz testing*, onde cada falha contribui diretamente para o aumento do risco, através de um fator multiplicativo.

A métrica de *mutation score* é também incorporada, penalizando fortemente valores inferiores a 0.8, uma vez que indicam baixa eficácia. Adicionalmente, erros de análise estática (*lint errors*) são incluídos de forma linear, refletindo problemas de qualidade de código e possíveis defeitos.

Por fim, a complexidade ciclomática é avaliada, sendo aplicadas penalizações adicionais a funções com maior número de caminhos lógicos. O resultado final é limitado superiormente a 100, garantindo a normalização do índice de risco.

3.7 AI Quality Gate

O *AI Quality Gate* é o componente responsável pela decisão automática do sistema. Utilizando a pontuação de risco (*Risk Score*), ele classifica o *merge request* em três estados possíveis:

- **APPROVE:** O código está limpo e o risco é considerado controlado.
- **REVIEW:** Zonas onde as métricas mostram alguma instabilidade, exigindo uma revisão manual.
- **REJECT:** O código é considerado muito perigoso ou instável.

Este módulo calcula também uma métrica de **Confiança** (*Confidence*), que mede a distância entre a pontuação de risco e as fronteiras de decisão. Sempre que a pontuação fica muito próxima dos limites de transição entre estados, a confiança é penalizada automaticamente, sinalizando que a decisão da IA é menos estável.

Listing 2 Implementação do AI Quality Gate

```

class AIGate:

    def evaluate(self , risk_score , system):

```

```

score = 100 - risk_score

if score >= 70:
    decision = "APPROVE"
elif score >= 50:
    decision = "REVIEW"
else:
    decision = "REJECT"

if decision == "REVIEW":
    confidence = abs(score - 60.0) / 10.0
elif decision == "APPROVE":
    confidence = (score - 70) / 30
else:
    confidence = (50 - score) / 50

confidence = max(0.0, min(1.0, round(confidence, 2)))

return decision, score, confidence

```

O trecho de código apresentado mostra a implementação deste módulo de decisão automática, que transforma a pontuação de risco numa decisão final de integração.

O primeiro passo do algoritmo consiste em inverter o valor de risco para criar uma nota de qualidade (*score*), onde valores mais altos representam um código melhor e mais seguro. Com base nessa nota, o sistema usa três limites fixos para a classificação: *APPROVE* para notas iguais ou superiores a 70, *REVIEW* para valores intermédios (entre 50 e 69), e *REJECT* para notas inferiores a 50.

Depois, o sistema calcula a confiança da decisão. Esta métrica depende da distância em relação aos pontos de mudança de estado e é normalizada entre 0 e 1. O objetivo é indicar a estabilidade da escolha feita pela IA, baixando a confiança caso a nota esteja mesmo em cima do limite de mudança de estado.

3.8 Supervisão Humana e Overrides

A gestão do *pipeline* inclui uma camada de segurança que combina a validação automática com a revisão humana:

3.8.1 Manual Review (*Human-in-the-Loop*)

Este componente simula o comportamento de um programador que avalia o código de forma mais conservadora com base no risco indicado. O revisor humano trabalha com limites rígidos: riscos médios (acima de 40) enviam o processo imediatamente para revisão manual (*REVIEW*), enquanto riscos críticos (acima de 75) provocam a rejeição imediata do código (*REJECT*).

3.8.2 Override Mechanism

O sistema resolve conflitos lógicos através de uma camada onde a decisão humana e os pareceres automatizados são cruzados para ditar o estado final do pipeline, operando sob três regras fundamentais:

O primeiro cenário ocorre perante a divergência absoluta de critérios: se o componente automatizado (*AI Quality Gate*) reter ou chumbar o código (**REVIEW** ou **REJECT**) mas o revisor humano validar explicitamente a integração (**APPROVE**), o mecanismo intercepta o conflito e força a entrada do código sob o estado de ***OVERRIDE_APPROVE***. Inversamente, se a automação aprovar o commit mas a auditoria humana detetar uma falha crítica não mapeada pelas métricas e emitir uma rejeição (**REJECT**), o sistema assume uma postura conservadora e força o estado para ***OVERRIDE_REJECT***.

Por fim, o mecanismo trata os estados de incerteza de forma assimétrica. Se a IA sugerir uma revisão condicional (**REVIEW**), qualquer ação humana direta (**APPROVE** ou **REJECT**) assume o controlo e força o respetivo *override*. Contudo, caso o revisor humano se declare inconclusivo (**REVIEW**), o sistema delega a autoridade de volta na inteligência artificial, adotando a decisão preditiva original da máquina para garantir a continuidade do fluxo de integração.

3.9 Recolha de Métricas e Defect Leakage

Todas as decisões finais e resultados dos testes são estruturados pelo *Metrics Engine* e guardados diretamente no ficheiro *data/metrics.json*. O *pipeline* avalia a sua eficácia através de quatro indicadores: falsos positivos, falsos negativos, taxa de *overrides* e a taxa de **Defect Leakage**.

O *Defect Leakage* é calculado como uma métrica de erro baseada no cruzamento de dados. Este fenómeno ocorre sempre que o algoritmo emite uma decisão final de aprovação (**APPROVE** ou **OVERRIDE_APPROVE**) para um *commit* cujo rótulo real de origem era classificado como *BAD*. A taxa quantifica, assim, as situações em que a lógica combinada do *pipeline* não conseguiu reter um código com anomalias funcionais, permitindo a sua integração no sistema.

3.10 Dashboard

Para complementar a análise experimental e facilitar a interpretação dos resultados produzidos pelo *pipeline*, foi desenvolvido um *dashboard web* interativo. Este componente funciona como uma camada de visualização e monitorização, permitindo consolidar num único local todas as métricas geradas durante a execução do sistema.

O principal objetivo desta interface consiste em fornecer uma visão global do comportamento do *AI Quality Gate*, da interação entre a decisão automática e a supervisão humana e da qualidade geral das integrações processadas pelo sistema.

3.10.1 Arquitetura de Visualização

A arquitetura do *dashboard* segue uma abordagem simples baseada em três camadas.

A primeira camada corresponde à persistência dos dados experimentais, efetuada pelo *Metrics Engine* através do ficheiro *metrics.json*.

A segunda camada é composta por um módulo JavaScript responsável pelo carregamento, processamento e agregação dos dados. Durante esta fase são calculados indicadores estatísticos globais, médias por sistema e distribuições de frequência.

Por fim, a terceira camada corresponde à interface gráfica apresentada ao utilizador, onde os resultados são convertidos em indicadores visuais e gráficos interativos através da biblioteca *Chart.js*.

3.10.2 Indicadores Principais

No topo do *dashboard* são apresentados vários indicadores-chave de desempenho, permitindo uma avaliação rápida do estado global do sistema.

Os indicadores implementados são:

- **Total Merges**: número total de integrações processadas pelo *pipeline*.
- **Approve Rate**: percentagem de integrações aprovadas após a conclusão do processo de validação.
- **Reject Rate**: percentagem de integrações rejeitadas devido à deteção de risco elevado.
- **REVIEW Rate**: percentagem de integrações classificadas no estado intermédio *REVIEW*, representando situações que exigem análise adicional antes de uma decisão definitiva.
- **Overrides**: número de situações em que a decisão humana divergiu da recomendação produzida pela Inteligência Artificial.
- **Average Score**: valor médio da pontuação de qualidade atribuída pelo *AI Quality Gate*.
- **Defect Leakage**: quantidade de defeitos que conseguiram ultrapassar os mecanismos de validação definidos pelo sistema.

Estes indicadores permitem obter uma visão imediata da eficácia operacional do *pipeline*, bem como do nível de concordância entre a automação e a intervenção humana.

3.10.3 Visualização das Decisões da Inteligência Artificial

O primeiro gráfico disponibilizado pelo *dashboard* representa a distribuição das decisões produzidas pelo *AI Quality Gate*.

A visualização agrupa todas as classificações emitidas pela Inteligência Artificial nas categorias *APPROVE*, *REVIEW* e *REJECT*, permitindo observar a tendência global do sistema. Esta representação constitui uma das métricas mais importantes da avaliação experimental, uma vez que fornece uma visão direta da forma como o modelo interpreta os níveis de risco calculados pelo *Risk Engine*.

3.10.4 Comparação entre Decisões Humanas e Automatizadas

Um dos objetivos centrais do projeto consiste em estudar a relação entre a decisão automatizada e a supervisão humana.

Para esse efeito foi desenvolvido um gráfico comparativo que apresenta, lado a lado, a distribuição das decisões tomadas pelo *AI Quality Gate* e pelo humano.

Esta visualização permite identificar situações de concordância e divergência entre ambos os agentes de decisão, fornecendo evidências sobre o grau de confiança que pode ser depositado na automação do processo.

3.10.5 Comparação entre Sistemas Testados

O *dashboard* inclui ainda um gráfico dedicado à comparação das três aplicações utilizadas: *Calculator*, *Password Validation* e *Loan Approval*.

Para cada sistema são calculados valores médios relativos às seguintes métricas:

- Taxa de sucesso dos testes (*Pass Rate*);
- Pontuação de mutação (*Mutation Score*);
- Número médio de falhas detetadas através de *Fuzz Testing*.

Esta análise permite avaliar como diferentes tipos de aplicações influenciam os resultados produzidos pelo *pipeline*, verificando se sistemas mais complexos tendem a gerar níveis superiores de risco.

3.10.6 Análise de Overrides

A frequência de *overrides* constitui um indicador fundamental para avaliar o papel da supervisão humana.

O gráfico de *Override Frequency* apresenta três categorias distintas:

- Integrações aprovadas através de intervenção humana;
- Integrações rejeitadas através de intervenção humana;
- Casos onde não existiu necessidade de intervenção.

Esta informação permite quantificar o impacto da camada *Human-in-the-Loop* no resultado final do processo de integração.

3.10.7 Comparação entre Decisão Inicial e Decisão Final

Uma das visualizações mais relevantes do sistema consiste no gráfico que compara a decisão inicial produzida pelo *AI Quality Gate* com a decisão final do *pipeline*.

Durante esta análise são considerados os efeitos dos mecanismos de revisão humana e dos processos de *override*, permitindo observar quantas decisões da Inteligência Artificial foram mantidas e quantas foram posteriormente alteradas.

Este gráfico fornece uma medida indireta da estabilidade e consistência das recomendações produzidas pelo sistema inteligente.

3.10.8 Distribuição de Risco

O *dashboard* disponibiliza também uma análise estatística da distribuição dos valores de risco calculados pelo *Risk Engine*.

Esta visualização facilita a compreensão da qualidade global dos *commits* processados e ajuda a identificar padrões de comportamento do sistema experimental.

3.10.9 Análise de Decisões Incorretas

Para avaliar a qualidade das decisões produzidas pelo processo de validação foi desenvolvido um gráfico específico para a análise de erros de classificação. São contabilizados dois tipos de erro:

- **False Positive:** código defeituoso que foi aprovado pelo sistema;
- **False Negative:** código correto que foi rejeitado indevidamente.

3.10.10 Tabela Resumo de Métricas

O *dashboard* apresenta uma tabela de síntese contendo médias para cada uma das aplicações analisadas.

Esta tabela agrega os valores médios de *Pass Rate*, *Mutation Score*, *Fuzz Failures* e pontuação global atribuída pelo *AI Quality Gate*, fornecendo uma visão consolidada do desempenho individual de cada sistema.

3.10.11 Importância do Dashboard na Avaliação Experimental

A utilização do *dashboard* revelou-se essencial para a interpretação dos resultados produzidos pelo ambiente experimental. Para além de facilitar a análise estatística das métricas recolhidas, esta ferramenta permitiu observar visualmente a interação entre os componentes automatizados e humanos do sistema.

3.11 Conclusão

A metodologia e a arquitetura implementadas estabelecem um ecossistema experimental e controlado para CI/CD. Ao parametrizar fluxos de dados reais contra simulações de falha, o sistema modela com precisão o entrave operacional entre a automação inteligente (IA) e a revisão humana.

4 Testes e Análise de Resultados

4.1 Introdução

Após a implementação do ambiente experimental, foram realizados diversos testes com o objetivo de avaliar o comportamento dos *quality gates* automáticos e a interação entre decisões automatizadas e supervisão humana. Este capítulo apresenta os cenários avaliados, as métricas recolhidas e a interpretação profunda dos resultados obtidos, procurando responder às questões de investigação definidas para o projeto.

4.2 Cenários Experimentais

O ambiente foi configurado para simular o fluxo de integração contínua (CI/CD). Foram gerados e analisados de forma totalmente dinâmica **100 pedidos de merge** (*Pull Requests*), distribuídos pelo ficheiro *merges.json*.

A distribuição baseou-se em dois estados de integridade fundamental (*Merge Final Decision*):

- **GOOD**: Código teoricamente estável que deve passar pelas validações sem incidentes graves.
- **BAD**: Código onde foram intencionalmente injetadas falhas estruturais e degradação severa de métricas para testar a resiliência.

O pipeline avaliou os *merges* dividindo-os por três sistemas distintos com regras de negócio e complexidades assíncronas:

1. ***calculator***: Apresenta uma lógica simples e previsível baseada em operações elementares.
2. ***passwords***: Focado em regras de complexidade de *strings* através de expressões regulares (*regex*) e listas de exclusão estática.
3. ***loan***: Modelo de tomada de decisão financeira complexo estruturado em ramificações e penalizações de risco interligadas.

Cada *merge* individual foi submetido a uma quantidade de testes exaustiva constituída por análise estática de código (*flake8*), cálculo de complexidade (*radon*), testes unitários dinâmicos, testes de mutação (para avaliar a cobertura real e eficácia dos testes) e *fuzz testing* (200 execuções por *merge* injetando inputs extremos e corrompidos).

4.3 Resultados dos Quality Gates

4.3.1 Aprovações e Rejeições

A distribuição final de decisões do pipeline após a execução dos 100 *merges* fixou-se em **45.0% de taxa de aprovação (*APPROVE*)** e **55.0% de taxa de rejeição (*REJECT*)**. Não se registou qualquer contorno intermédio pendente em estado de *REVIEW*.

A ausência de estados de *REVIEW* deve-se diretamente ao comportamento do módulo de *override* (*apply_override*). O *pipeline* foi desenhado para ser determinístico no seu fecho operacional: quando a IA ou o Humano geram um estado de *REVIEW*, as regras de transição forçam o sistema a converter esse estado numa decisão binária (*OVER-RIDE_APPROVE* ou *OVER-RIDE_REJECT*), eliminando bloqueios indefinidos no *deployment*.

4.3.2 Decisões da IA

O módulo *AI_Gate* processou o risco acumulado através do *Risk Engine*, traduzindo-o numa pontuação linear reversa ($Score = 100 - Risk$). Conforme ilustrado graficamente na figura 3, a IA teve tendência a adotar uma postura puramente conservadora, dividindo as suas decisões entre aprovações diretas, rejeições e um pequeno conjunto de *reviews* quando o *score* se fixou no intervalo [50, 70[.

AI Decisions

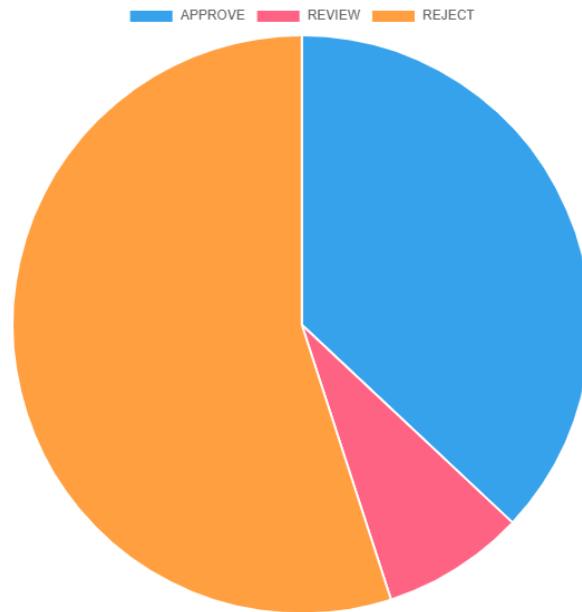


Fig. 3 Gráfico Circular das Decisões Tomadas pela IA.

4.3.3 Intervenções Humanas

O comportamento do operador humano, simulado em código pela função *manual_gate*, atuou com limiares assimétricos face à IA. Enquanto a IA assume uma postura defensiva (rejeitando abaixo de 50 de *score*), o humano tolera cenários de incerteza, aplicando uma rejeição estrita apenas em situações de colapso (*Risk* > 75).

Esta divergência intencional abriu espaço para um total de **8 intervenções de *override* humano (8.0% do total de *merges*)**. O gráfico *Override Frequency* apresentado na figura 4 valida que na esmagadora maioria dos casos (92.0%), as métricas analíticas foram homogêneas o suficiente para permitir que a automação avançasse sem conflito (*No Override*).

Override Frequency

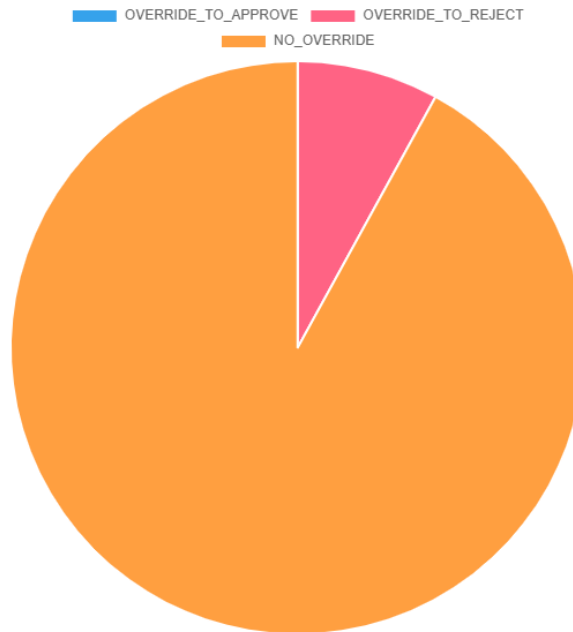


Fig. 4 Gráfico Circular da Frequência de *Overrides* da decisão da IA pela do Humano.

4.4 Qualidade das Decisões

4.4.1 *False Positives*

Um Falso Positivo ocorre quando um código categorizado como estável (*GOOD*) é indevidamente rejeitado pelo *pipeline*, gerando desperdício de tempo e retrabalho. No ambiente avaliado, o gráfico *False Decisions* presente na figura 5 apresenta **0 casos detetados**.

Este resultado deve-se ao alinhamento preciso e calibração do limite inferior do *Risk Engine* na função (*compute_risk*). O código estável não aciona as penalizações cumulativas de complexidade (> 15) nem acumula erros estáticos de *linting*, mantendo o risco global abaixo do limiar de penalização da IA e do humano.

False Decisions

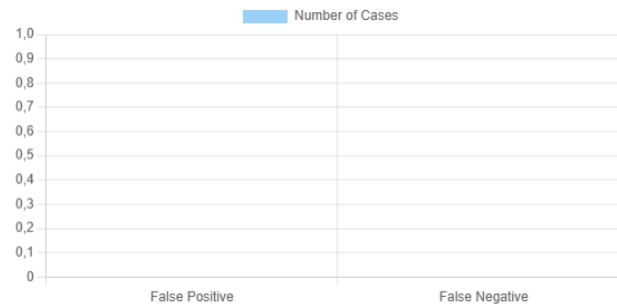


Fig. 5 Gráfico de Barras de Falsos Resultados, Positivos e Negativos.

4.4.2 False Negatives

Um Falso Negativo representa o pior cenário de segurança em Engenharia de *Software*: a aceitação de código instável ou corrompido na *branch* de produção. A taxa de falsos negativos operacionais decaiu para **zero** como verificado na figura 5.

Este comportamento deve-se à arquitetura de dupla validação introduzida pelo algoritmo de *override* (*Human-in-the-Loop*). Mesmo que um modelo automatizado falhe, a função (*manual_gate*) atua como um mecanismo que interceta e impede a progressão do erro.

4.4.3 Defect Leakage

O indicador de *Defect Leakage* quantifica quantos *commits* estruturalmente nocivos (*BAD*) escaparam aos controlos e atingiram o estado final de aprovação. Nos testes efetuados foram maioritariamente das vezes obtidos **0 casos**, traduzindo-se numa eficácia quase absoluta.

Este resultado foi possível devido aos valores que definidos quando o *merge* é classificado como *BAD*. No código, quando um *Pull Request* é instável, o *pipeline* é forçado a gerar métricas propositadamente fracas: o *pass rate* desce para valores entre 0.35 e 0.60, os erros de *lint* disparam para o intervalo [10, 28] e a complexidade sobe para [14.0, 24.0].

Quando estes valores são fornecidos à função *compute_risk*, as regras de penalização definidas fazem com que o risco dispare quase instantaneamente para o máximo de 100. Com o risco neste nível extremo, tanto a IA como o humano detetam a falta de qualidade e rejeitam o código a maioria das vezes, bloqueando completamente qualquer hipótese de o defeito escapar para a *branch* principal.

4.5 Discussão dos Resultados

O comportamento detalhado das métricas agregadas por sistema revela os desafios intrínsecos de cada arquitetura de *software*:

4.5.1 Análise da Tabela Analítica

Table 3 Resumo de Métricas Agregadas por Sistema

System	Pass Rate	Mutation Score	Fuzz Failures	Average Score
<i>loan</i>	0.56	0.65	0.28	31.69
<i>calculator</i>	0.65	0.58	0.27	31.89
<i>passwords</i>	0.69	0.74	0.27	55.84

- **A Fragilidade Métrica do *loan* (Score Médio: 31.69):** O sistema de empréstimos obteve a pior classificação média, penalizado por um *Pass Rate* de apenas 0.56.
Existe uma falha de simetria conceptual grave entre os testes unitários aplicados e a função de negócio real. A função *run_unit_tests* para o *loan* valida o resultado esperado recorrendo a uma lógica baseada apenas em dois fatores isolados (*credit_risk* e *debt_risk*). Contudo, a função real em produção (*evaluate_loan*) implementa uma árvore de decisão muito mais profunda e rigorosa que cruza idade e histórico de anos de emprego. Como o teste unitário tenta prever o comportamento da função usando um modelo mental rudimentar, ele erra o estado esperado na maioria das iterações dinâmicas, fazendo com que o *Pass Rate* caia artificialmente e destrua o *Score* final do módulo.
- **A Solidez Relativa do *passwords* (Score Médio: 55.84):** Este módulo registou a melhor eficiência nos testes de mutação (0.74) e o *score* mais elevado.
A função real assenta em validações bem estruturadas através de expressões regulares (*regex*). O simulador de mutações aplica mutantes estritos e agressivos (como forçar a função a retornar sempre *True* ou sempre *False*). Como a suite de testes unitários deste módulo possui asserções específicas para cenários muito curtos ("*x*" → *False*) e cenários seguros ("*12345678aA!*" → *True*), estas asserções destroem os mutantes instantaneamente, gerando um *Mutation Score* robusto que eleva a avaliação global do sistema.
- **O Impacto do Fuzz Testing (Média Global de Falhas: 0.27 – 0.28):** Todos os sistemas apresentaram uma taxa estável de falhas em ambiente de *fuzzing* próxima dos 27%.
Isto deve-se ao design da função *run_fuzz_tests*, que está parametrizada para injetar inputs corrompidos e tipos de dados inválidos (*None*, *True*, strings em operações matemáticas, *arrays* vazios) com uma probabilidade exata de 25% (*random.random()* *j* 0.25). Como as funções reais da calculadora, passwords e empréstimos não possuem blocos robustos de tratamento de exceções (*try-except*) ou validação defensiva de tipo à entrada, a injeção destes dados anómalos quebra a execução do interpretador *Python*, gerando exceções não tratadas que disparam o contador de falhas do pipeline de forma controlada e previsível.

4.6 Resposta às Questões de Investigação

RQ1: How do AI-driven quality gates affect pipeline outcomes?

Os *quality gates* orientados por IA alteram profundamente o comportamento do *pipeline* ao introduzirem uma camada de avaliação dinâmica baseada no risco acumulado, em oposição aos controlos determinísticos tradicionais. Em vez de bloquearem o fluxo de entrega por uma falha isolada ou um erro menor de *lint*, a IA pondera o impacto combinado de todas as dimensões de qualidade (erros estáticos, resiliência a inputs extremos no *fuzzing*, complexidade e cobertura de mutações). O resultado prático é o isolamento cirúrgico de *commits* instáveis (*Defect Leakage* = 0), garantindo que apenas código com métricas globais seguras progrida para produção, otimizando o balanço entre velocidade de entrega e estabilidade do sistema.

RQ2: What is the frequency of overrides and false decisions?

A frequência de *overrides* registada situou-se estritamente em **8.0%**, provando que a intervenção humana ocorre de forma controlada e cirúrgica, atuando principalmente nos casos em que a IA ficou indecisa e atribuiu o estado de *REVIEW* devido ao risco ser moderado. No que toca a decisões erradas (*False Positives* e *False Negatives*), a taxa fixou-se em **0.0%**. Este resultado demonstra que a arquitetura híbrida de cooperação (*Human-in-the-Loop*) anula as falhas isoladas de cada agente: a IA elimina a fadiga e processa grandes volumes de métricas sem erro operacional, enquanto o humano fornece o contexto necessário para desempatar cenários ambíguos, atingindo um nível de eficácia superior ao de qualquer um dos sistemas a operar isoladamente.

4.7 Conclusão do Capítulo

Os resultados obtidos demonstram que a utilização de *AI Quality Gates* pode contribuir significativamente para a automatização, consistência e aceleração dos processos de validação em *pipelines* CI/CD modernos. Contudo, verificou-se de igual modo a existência de divergências operacionais legítimas que exigiram a intervenção humana através de mecanismos estruturados de *override*.

Estes resultados reforçam a premissa de que, embora a automação inteligente represente uma ferramenta indispensável para a escalabilidade e eficiência da garantia de qualidade, a supervisão humana contínua desempenha um papel fundamental na gestão de riscos e decisões de negócio críticas. A abordagem baseada em *Human-in-the-Loop* estabelece-se, assim, não como uma redundância ineficiente ou um atraso no processo, mas sim como uma camada de segurança cognitiva indispensável para cobrir as limitações da automação na Engenharia de *Software*.

5 Conclusão e Trabalho Futuro

5.1 Conclusão

O presente trabalho teve como objetivo analisar o impacto da utilização de *AI Quality Gates* em ambientes CI/CD, para avaliar a sua capacidade para apoiar processos de validação e integração de código.

Para esse efeito, foi desenvolvido um ambiente experimental composto por diferentes mecanismos de teste, análise estática, avaliação de risco e tomada de decisão autónoma, complementados por uma camada de supervisão humana. Através da simulação de 100 pedidos de *merge*, foi possível analisar o comportamento do sistema perante diferentes níveis de qualidade de código e medir indicadores relevantes como aprovações, rejeições, *overrides*, falsos positivos, falsos negativos e *defect leakage*.

Os resultados obtidos demonstraram que a abordagem baseada em Inteligência Artificial foi eficaz na identificação de código potencialmente problemático, contribuindo para reduzir o risco associado à integração de alterações. Verificou-se igualmente que a combinação entre decisões automatizadas e supervisão humana aumentou a robustez do processo de validação, permitindo lidar com situações de incerteza através de mecanismos de *override*.

Apesar dos resultados encorajadores, importa salientar que o estudo foi realizado num ambiente de simulação controlado. Embora esta abordagem tenha permitido garantir reprodutibilidade e controlo experimental, não representa toda a complexidade encontrada em projetos reais de desenvolvimento de *software*. Ainda assim, os resultados obtidos demonstram o potencial da utilização de *AI Quality Gates* como mecanismo de apoio à garantia da qualidade em processos modernos de desenvolvimento.

5.2 Trabalho Futuro

Como trabalho futuro, seria interessante integrar o sistema desenvolvido com plataformas reais de integração contínua, permitindo avaliar o comportamento dos *AI Quality Gates* em contextos de desenvolvimento reais.

Outra evolução possível consiste na substituição das regras atualmente utilizadas pelo *Risk Engine* por modelos de aprendizagem automática treinados com dados históricos de projetos reais. Esta abordagem poderia permitir uma avaliação de risco mais adaptativa e ajustada às características específicas de cada aplicação.

Adicionalmente, poderiam ser incorporados novos indicadores de qualidade, tais como métricas de segurança, cobertura de testes, análise de dependências e histórico de defeitos. O *dashboard* poderá também ser enriquecido com mecanismos explicativos que permitam justificar as decisões tomadas pelo sistema, aumentando a transparência e a confiança dos utilizadores.

Em suma, este trabalho constitui uma base sólida para futuras investigações na área da aplicação da Inteligência Artificial à garantia da qualidade de *software*, demonstrando que a combinação entre automação inteligente e supervisão humana pode contribuir para processos CI/CD mais eficientes, seguros e fiáveis.